

Estudo Dirigido · Guia 5

Persistência de Dados

Ranking com JSON — Salvando o Hall da Fama

Utilizaremos o módulo embutido `json` do Python para salvar e carregar a lista de pontuações no formato JSON.

Objetivos de Aprendizagem

- Compreender o formato JSON e sua estrutura de dados
- Usar `json.dump()` para salvar o ranking em disco
- Usar `json.load()` para carregar o ranking ao iniciar o jogo
- Tratar erros de arquivo inexistente e JSON corrompido
- Implementar o fluxo completo de atualização do Top 10

1. O Formato JSON

JSON (JavaScript Object Notation) é um formato de texto leve, legível por humanos e máquinas, amplamente usado para troca de dados. No Python, o módulo embutido `json` converte dicionários e listas em texto JSON — e vice-versa — de forma simples e eficiente.

O ranking do jogo será armazenado como uma `lista de dicionários`, onde cada dicionário representa uma entrada do jogador:

```
# Estrutura do ranking (conforme ANEXO II do projeto)
[
    {
        "nome":      "Yoda",
        "pontuacao": 2500,
        "data":      "2026-03-18 10:30:00"
    },
    {
        "nome":      "Luke",
        "pontuacao": 1800,
        "data":      "2026-03-17 22:15:44"
    }
]
```



Por que JSON?

JSON é texto puro — pode ser aberto no Bloco de Notas, editado manualmente e versionado com Git. O Python lê e escreve arquivos JSON com apenas duas funções: `json.dump()` e `json.load()`.

2. Salvar Dados — `json.dump()`

2.1. A Função `save_ranking()`

Use `json.dump()` para gravar a estrutura de dados em disco. Sempre envolva a operação em `try/except` para tratar erros de I/O de forma robusta:

```
import json
import os

RANKING_FILE = "hall_da_fama.json"

def save_ranking(top_scores_data):
    """
    Salva a lista de melhores pontuações em um arquivo JSON.

    :param top_scores_data: Lista de dicionários com nome,
                             pontuacao e data/hora.
    """
    try:
        # 'w'      → cria ou sobrescreve o arquivo
        # utf-8    → suporta acentos e caracteres especiais
        # indent=4 → formata o JSON legível por humanos
        with open(RANKING_FILE, 'w', encoding='utf-8') as f:
            json.dump({"top_scores": top_scores_data}, f, indent=4)
        print(f"Ranking salvo com sucesso em {RANKING_FILE}")
    except IOError as e:
        print(f"Erro ao salvar o arquivo de ranking: {e}")
```

⚠ Atenção ao parâmetro `indent`

`indent=4` é opcional mas recomendado durante o desenvolvimento, pois torna o arquivo legível. Em produção, você pode omiti-lo para gerar um JSON mais compacto (sem espaços extras).

3. Carregar Dados — `json.load()`

3.1. A Função `load_ranking()`

Use `json.load()` para ler o ranking ao iniciar o jogo. Sempre verifique se o arquivo existe antes de abri-lo e trate os dois erros mais comuns:

Exceção	Causa	Ação Recomendada
<code>json.JSONDecodeError</code>	Arquivo existe mas está corrompido ou vazio	Retornar lista vazia e registrar o erro
<code>IOError / OSError</code>	Falha de permissão ou disco indisponível	Retornar lista vazia e registrar o erro

```
def load_ranking():
    """
    Carrega o ranking do arquivo JSON.
    Se o arquivo não existir ou estiver corrompido,
    retorna uma lista vazia.

    :return: Lista de dicionários com as pontuações.
    """
```

```
# 1. Verifica se o arquivo existe (primeira execução do jogo)
if not os.path.exists(RANKING_FILE):
    print(f"{RANKING_FILE} não encontrado. Iniciando ranking vazio.")
    return []

# 2. Tenta carregar e decodificar o JSON
try:
    with open(RANKING_FILE, 'r', encoding='utf-8') as f:
        data = json.load(f)
        # Extrai apenas a lista de scores do objeto raiz
        return data.get("top_scores", [])

except json.JSONDecodeError as e:
    print(f"JSON corrompido em {RANKING_FILE}: {e}")
    return []

except IOError as e:
    print(f"Erro de I/O ao carregar ranking: {e}")
    return []
```

Por que data.get("top_scores", [])?

O ranking é salvo dentro de um objeto raiz: {"top_scores": [...]}. O método .get() retorna a lista ou [] caso a chave não exista, evitando que uma KeyError quebre o jogo em arquivos com formato inesperado.

4. Atualização e Fluxo Completo do Ranking

Após o Game Over, o jogo deve executar os seguintes passos para garantir que o ranking esteja sempre atualizado e limitado aos 10 melhores jogadores:

Passo	Ação	Detalhes
1	Carregar	Chamar load_ranking() para obter a lista atual do disco.
2	Nova entrada	Criar dicionário com nome, pontuacao e datetime.now().
3	Inserir	Adicionar o novo dicionário à lista com list.append().
4	Ordenar	Reordenar pela chave 'pontuacao' em ordem decrescente (reverse=True).
5	Limitar	Fatiar a lista: top_scores = top_scores[:10] — mantém só os 10 melhores.
6	Salvar	Chamar save_ranking(top_scores) para persistir o resultado.

4.1. Implementação Completa

```
from datetime import datetime

def update_ranking(player_name, score):
    """Atualiza o ranking com uma nova pontuação após o Game Over."""

    # Passo 1 — Carregar ranking atual
```

```
top_scores = load_ranking()

# Passo 2 - Criar nova entrada
new_entry = {
    "nome":      player_name,
    "pontuacao": score,
    "data":      datetime.now().strftime("%Y-%m-%d %H:%M:%S")
}

# Passo 3 - Inserir na lista
top_scores.append(new_entry)

# Passo 4 - Ordenar do maior para o menor
top_scores.sort(key=lambda x: x["pontuacao"], reverse=True)

# Passo 5 - Manter apenas o Top 10
top_scores = top_scores[:10]

# Passo 6 - Salvar em disco
save_ranking(top_scores)

# — Chamada típica no Game Over —————
# update_ranking(player_name="Yoda", score=2500)
```

✓ Resultado Esperado

O arquivo `hall_da_fama.json` será criado (ou atualizado) na pasta do jogo, contendo sempre os 10 melhores resultados ordenados, persistidos entre sessões.

5. Referência de Funções JSON

Função / Método	Descrição
<code>json.dump(obj, f, indent=4)</code>	Serializa <code>obj</code> (dict/list) em texto JSON e escreve no arquivo <code>f</code> .
<code>json.dumps(obj)</code>	Serializa <code>obj</code> em uma string JSON (sem arquivo).
<code>json.load(f)</code>	Lê o arquivo <code>f</code> e retorna o objeto Python correspondente.
<code>json.loads(string)</code>	Converte uma string JSON em objeto Python.
<code>dict.get(chave, padrao)</code>	Retorna o valor da chave ou padrão se ela não existir.
<code>os.path.exists(caminho)</code>	Retorna <code>True</code> se o arquivo/pasta existir no caminho especificado.
<code>datetime.now().strftime(fmt)</code>	Retorna a data/hora atual formatada como string (ex: '%Y-%m-%d %H:%M:%S').

6. Exercícios Práticos

Consolide os conceitos com os desafios abaixo:

Exercício 1 — Primeiro arquivo JSON

Crie um script separado que gere manualmente um arquivo `ranking_teste.json` com 3 entradas fictícias usando `json.dump()`. Abra o arquivo gerado em um editor de texto e verifique a formatação.

Exercício 2 — Leitura segura

Implemente `load_ranking()` e teste os três cenários: (a) arquivo inexistente, (b) arquivo válido com dados, (c) arquivo com JSON inválido (edite o `.json` manualmente e apague uma chave).

Exercício 3 — Simulação de partidas

Crie um loop que simule 15 partidas com nomes e pontuações aleatórias (use o módulo `random`). Chame `update_ranking()` para cada uma e exiba o Top 10 final no terminal.

Exercício 4 — Empate na pontuação

O que acontece quando dois jogadores têm a mesma pontuação? Adapte o `sort()` para usar `data/hora` como critério de desempate — a partida mais recente deve aparecer primeiro.

Exercício 5 — Integração com o jogo

Localize a tela de Game Over no seu jogo e integre a função `update_ranking()`. Peça o nome do jogador com `pygame` e chame a função ao confirmar. Exiba o Top 5 na própria tela de Game Over.

7. Próximos Passos

Com a persistência de dados implementada, o jogo já mantém o Hall da Fama entre sessões. Os próximos guias completarão a experiência:

Guia 6 — HUD e Interface

- `pygame.font` para placar e vidas na tela
- Exibição do Top 5 na tela de Game Over
- Barra de progresso e indicadores visuais

Guia 7 — Estados de Jogo

- Telas de Menu, Jogo, Pausa e Vitória
- Máquina de estados com dicionários
- Integração completa do ranking na UI